

Assignment 6 solution

1. 【考点：算法的基本概念】下列程序段的时间复杂度为（ ）

```
int i = 1, j = 0;
while (i < n) {
    j++;
    i = i * 2;
}
```

- A. $O(n)$
- B. $O(\log_2 n)$
- C. $O(n \log_2 n)$
- D. $O(1)$

【答案】：B

【解析】：每次循环 i 的值乘以 2，当 $2^k \geq n$ 时循环结束，其中 k 为循环次数。解得 $k = \lceil \log_2 n \rceil$ ，故时间复杂度为 $O(\log_2 n)$

2. 【考点：栈和队列的基本概念】设栈的初始状态为空，当字符序列 a, b, c, d, e, f 依次入栈时，下列哪个序列不可能是合法的出栈序列？（ ）

- A. b, c, a, e, f, d
- B. c, b, d, a, f, e
- C. d, e, c, f, b, a
- D. a, d, e, f, b, c

【答案】：D

【解析】：出栈序列的一个重要性质是：对于序列中任何一个出栈元素，在它之后出栈的且比它先入栈的元素，必须是逆序的。D 选项中， d 出栈后，比 d 先入栈的 b, c 出栈顺序为 b, c （正序），这是不可能的（栈底到栈顶为 b, c ，出栈应为 c, b ）

3. 【考点：二叉树的链式存储结构】在一棵具有 n 个结点的非空二叉树中，采用二叉链表作为存储结构，其中空指针域的个数为（ ）

- A. n
- B. $n+1$
- C. $n-1$
- D. $2n$

【答案】：B

【解析】：每个结点有 2 个指针域，总共有 $2n$ 个指针域。除了根结点外，每个结点都被一个非空指针指向，即有 $n-1$ 个非空指针。因此空指针数量为 $2n - (n - 1) = n + 1$

4. 【考点：图的基本概念】已知无向图 G 含有 16 条边，其中度为 4 的顶点个数为 3，度为 3 的顶点个数为 4，其余顶点的度均为 2。则图 G 的顶点总数为（ ）

- A. 11
- B. 13
- C. 15
- D. 17

【答案】：A

【解析】：根据图论基本定理（握手定理），所有顶点的度数之和等于边数的 2 倍。设度为 2 的顶点有 x 个，则有 $4 \times 3 + 3 \times 4 + 2 \times x = 16 \times 2$ 。解得 $24 + 2x = 32 \Rightarrow x = 4$ 。顶点总数为 $3 + 4 + 4 = 11$ 个

5. 【考点：二叉树搜索树】在一棵严格的二叉排序树（BST）中，若某结点 P 有且仅有一个左孩子结点 L，且没有右孩子。则以下关于中序遍历该树的结论必然正确的是（ ）

- A. 结点 P 是结点 L 的直接前驱
- B. 结点 L 是结点 P 的直接前驱
- C. 结点 L 是结点 P 的直接后继
- D. 结点 P 是全树最大的结点

【答案】：B

【解析】：中序遍历顺序为“左-根-右”。因为 P 没有右孩子，所以遍历完 P 的左子树（其中最后一个遍历的是 L 的最右下代，若 L 无右子树则是 L 本身）后，必定紧接着遍历 P。所以 L（或 L 子树中的最大结点）是 P 的直接前驱

6. 【考点：顺序查找法】对长度为 n 的有序单链表进行查找，若采用顺序查找，其平均查找长度为（ ）

- A. $O(1)$
- B. $O(\log_2 n)$
- C. $O(n)$
- D. $O(n \log_2 n)$

【答案】：C

【解析】：单链表由于不支持随机访问，即使是有序的，也无法进行折半查找，只能从头到尾进行顺序查找。最坏查找 n 次，平均查找 $(n+1)/2$ 次，时间复杂度均为 $O(n)$

7. 【考点：图的遍历】采用邻接表存储的有向图进行广度优先搜索（BFS），其空间复杂度为（ ）

- A. $O(1)$
- B. $O(|V|)$
- C. $O(|E|)$
- D. $O(|V| + |E|)$

【答案】：B

【解析】：BFS 需要借助一个辅助队列来存放被访问的顶点，最坏情况下图中所有顶点都在同一层，队列中最多有 $|V|$ 个元素，此外还需要一个 `visited` 数组记录访问状态，大小也为 $|V|$ 。因此空间复杂度为 $O(|V|)$

8. 【考点：二叉树的定义及其主要特征】若一棵完全二叉树有 768 个结点，则该二叉树中叶子结点的个数是（ ）

- A. 383
- B. 384
- C. 385
- D. 386

【答案】：B

【解析】：对于完全二叉树，若结点总数 n 为偶数，则度为 1 的结点数 $n_1 = 1$ ；若 n 为奇数，则 $n_1 = 0$ 。本题 $n=768$ （偶数），故 $n_1 = 1$ 。由 $n_0 + n_1 + n_2 = n$ 且 $n_0 = n_2 + 1$ 可推导出 $n = 2n_0 + n_1 - 1$ 。代入得 $768 = 2n_0 + 1 - 1$ ，解得 $n_0 = 384$

9. 【考点：栈和队列的顺序存储结构】循环队列存储在数组 `A[0..n-1]` 中，其头尾指针分别为 `front` 和 `rear`，则出队时头指针的修改语句是（ ）

- A. `front = front + 1`
- B. `front = (front + 1) % (n - 1)`
- C. `front = (front + 1) % n`
- D. `front = (front - 1) % n`

【答案】：C

【解析】：循环队列由于是环状数组，指针加 1 后必须对数组长度 n 取余以实现“折返”，正确的自增逻辑是 `(front + 1) % n`

10. 【考点：排序算法的分析和应用】下列排序算法中，元素的移动次数与初始序列的排列顺序无关的是（ ）
- A. 直接插入排序
 - B. 冒泡排序
 - C. 基数排序
 - D. 快速排序

【答案】：C

【解析】：基数排序是通过分配和收集来实现排序的，无论初始序列如何，都需要进行 d 趟分配和收集，元素的移动（从原数组到桶，再从桶回原数组）次数完全固定。插入、冒泡、快排的移动次数都与初始状态高度相关

11. 【考点：平衡二叉树、红黑树】平衡二叉树（AVL树）和红黑树（Red-Black Tree）都是高级的自平衡二叉搜索树结构。下列关于这两种数据结构的性质与效率推导中，正确的组合是（ ）
- I. 若一棵红黑树的所有内部结点（非空结点）均为黑色，则该树一定是一棵满二叉树（即完美二叉树，所有叶子都在同一层）
 - II. 任何一棵红黑树，必定同时也是一棵平衡二叉树（AVL树），即满足所有结点的左右子树高度差不超过 1
 - III. 在含有 n 个结点的 AVL 树和红黑树中分别插入一个新结点，为了恢复各自的结构平衡，在最坏情况下，两者所需的“旋转”操作次数均为 $O(1)$ 常数级别
 - IV. 在含有 n 个结点的 AVL 树和红黑树中分别删除一个结点，为了恢复各自的结构平衡，在最坏情况下，两者所需的“旋转”操作次数均为 $O(1)$ 常数级别
- A. 仅 I、II
 - B. 仅 I、III
 - C. 仅 II、IV
 - D. 仅 I、III、IV

【答案】：B

【解析】：

I 正确：红黑树的核心性质之一是“从任意结点到其每个叶子结点（NIL 空结点）的所有路径都包含相同数目的黑色结点”（即黑高相同）。如果树中所有内部结点都是黑色，意味着到任何一个 NIL 结点的实际路径长度必须完全相等。这就要求该树的底层必须被完全填满，即它一定是一棵满二叉树（Perfect Binary Tree）

II 错误：红黑树的平衡条件比 AVL 树宽松得多。红黑树只保证“最长路径（红黑交替）不超过最短路径（全黑）的两倍”，因此其结点的左右子树高度差完全有可能超过 1，它并不一定满足 AVL 树严格的高度差定义

III 正确：

AVL 树插入：插入破坏平衡后，只需找到最小不平衡子树的根结点，进行 1 次单旋转或 1 次双旋转（最多 2 次旋转动作），即可恢复平衡，且恢复后的子树高度与插入前完全一致，不会再向上波及。故最坏旋转次数为 $O(1)$

红黑树插入：插入后可能引发颜色的向上传递（变色最多 $O(\log n)$ 次），但真正涉及结构的“旋转”操作最多只需要 2 次，之后树就必定满足所有性质。故最坏旋转次数也为 $O(1)$

IV 错误：

红黑树删除：删除结点后，引起的失衡通过最多 3 次旋转即可完全恢复，旋转次数为 $O(1)$

AVL 树删除：删除结点可能导致某棵子树高度变矮。当对最小不平衡子树进行旋转恢复后，该

子树整体高度依然可能比删除前矮，这会导致更高层的结点继续失衡，从而发生“连锁反应”。在最坏情况下，旋转操作可能一路向上回溯到根结点，需要 $O(\log n)$ 次旋转。因此，两者在删除时的最坏旋转次数并不相同

12. 【考点：B 树及其基本操作】设一棵 m 阶 ($m \geq 3$) 的 B 树，含有 N 个关键字。关于该 B 树，下列叙述正确的是 ()

- A. 树中所有非叶子结点的子树个数必然等于它所包含的关键字个数加 1
- B. 根结点至少有两个孩子结点
- C. 该树的高度 h (不含外部叶子空结点层) 的最大值为 $\log_{\lceil m/2 \rceil} ((N+1)/2) + 1$
- D. 每个非根的内部结点至少含有 $\lfloor m/2 \rfloor$ 个关键字

【答案】：C

【解析】：A 错，所有非叶子结点的子树个数等于关键字个数加 1 是对的，但题目没有说是“内部结点”，叶子结点没有子树，所以 A 描述不严谨。B 错，如果树只有根结点，则根结点没有孩子。D 错，至少应含有 $\lceil m/2 \rceil - 1$ 个关键字。C 对，这是 B 树最大高度的经典推导公式

13. 【考点：关键路径】在一个有向无环图 (AOE 网) 中，若存在多条关键路径，则下列说法正确的是 ()

- A. 缩短任意一条关键路径上的活动持续时间，必然能够缩短整个工程的工期
- B. 关键路径上的活动称为关键活动，关键活动的最早开始时间必然小于最迟开始时间
- C. 只有将所有关键路径上共有的 (即交集) 关键活动的时间缩短，才有可能缩短整个工期
- D. 增加非关键路径上某个活动的持续时间，不会改变整个工程的关键路径

【答案】：C

【解析】：A 错，若有多条关键路径，只缩短其中一条，另一条仍然决定原工期，总工期不会缩短。B 错，关键活动的特征就是最早开始时间等于最迟开始时间 (即没有机动时间 $d = 0$)。D 错，如果非关键路径增加的时间使得它的长度超过了原来的关键路径，它就会变成新的关键路径。C 正确，缩短公共的关键活动能同时缩短所有关键路径，从而缩短工期

14. 【考点：外部排序】采用 k 路平衡归并和败者树技术进行外部排序，若增加归并路数 k ，在不考虑内存限制的情况下，下列各项中一定会减少的是 ()

- A. 内部归并时的总比较次数
- B. 读写外存的 I/O 次数
- C. 构建初始归并段的时间
- D. 败者树的高度

【答案】：B

【解析】：外部排序的总 I/O 次数主要由归并趟数决定，归并趟数 $S = \lceil \log_k (m) \rceil$ (m 为初始归并段数量)。增大 k 必定导致 S 减小，从而直接减少外存 I/O 次数。使用败者树后，内部比较次数与 k 无关，A 错。C 与 k 无关。败者树高度增加 (高度为 $\lceil \log_2 k \rceil$)，D 错

15. 【考点：字符串模式匹配】已知模式串 $T = \text{"ababaa"}$ ，利用 KMP 算法进行匹配时，求出的 `next` 数组 (下标从 1 开始记) 和 `nextval` 数组分别为 ()

- A. 0 1 1 2 3 4 和 0 1 0 1 0 4
- B. 0 1 1 2 3 4 和 0 1 1 1 0 4
- C. -1 0 0 1 2 3 和 -1 0 -1 0 -1 3
- D. 0 1 1 2 2 3 和 0 1 0 1 0 1

【答案】：A

【解析】：`next[1]=0`

串 T : a b a b a a

`next`: 0 1 1 2 3 4 (根据前缀和后缀最长相等长度 + 1 求得)

求 `nextval`:

`j=1: a, nextval[1]=0`

```

j=2: b, next[2]=1(a), b!=a, nextval[2]=next[2]=1
j=3: a, next[3]=1(a), a==a, nextval[3]=nextval[1]=0
j=4: b, next[4]=2(b), b==b, nextval[4]=nextval[2]=1
j=5: a, next[5]=3(a), a==a, nextval[5]=nextval[3]=0
j=6: a, next[6]=4(b), a!=b, nextval[6]=next[6]=4
结果: 0 1 0 1 0 4

```

16. 【考点：并查集及其应用】在包含 n 个元素的并查集中，为了提高查找效率采用了“按秩合并 (Union by Rank)”和“路径压缩 (Path Compression)”的优化策略。最坏情况下，执行 m 次 ($m \geq n$) 查找 (Find) 和合并 (Union) 操作的总时间复杂度是 ()

A. $O(m \log_2 n)$
 B. $O(n \log_2 n)$
 C. $O(m \alpha(n))$
 D. $O(m)$

【答案】：C

【解析】：若仅采用一种优化，单次操作复杂度可能接近 $O(\log N)$ 。当同时使用“按秩合并”和“路径压缩”时，单次操作的平摊时间复杂度为极其缓慢增长的反阿克曼函数 $\alpha(n)$ 。执行 m 次操作的总复杂度为 $O(m \alpha(n))$

17. 【考点：快速排序】设待排序的元素序列为 {12, 2, 16, 30, 28, 10, 16*, 20, 6, 18}，若采用某种排序算法进行了一趟排序后，结果变为了 {6, 2, 10, 12, 28, 30, 16*, 20, 16, 18}，则采用的排序算法最有可能是 ()

A. 直接插入排序
 B. 简单选择排序
 C. 快速排序
 D. 堆排序

【答案】：C

【解析】：观察排序后的序列，元素 12 的左边均小于 12 (6, 2, 10)，右边均大于等于 12 (28, 30, 16*, 20, 16, 18)。这恰好符合快速排序一趟划分后的特征 (12 作为枢轴被放到了最终位置)。注意 16 和 16* 的相对位置反了，这也验证了快排的不稳定性

18. 【考点：散列 (Hash) 表】对含有 N 个元素的哈希表，采用二次探测再散列法 (Quadratic Probing) 解决冲突。设哈希表长为 M ，且 M 为素数。为了保证每次探测都能找到一个空槽 (假设表未满)，必须满足的条件是 ()

A. 装填因子 $\alpha \leq 0.5$
 B. 装填因子 $\alpha \leq 0.75$
 C. 探测步长 $d_i = 1^2, -1^2, 2^2, -2^2 \dots$
 D. 哈希函数产生均匀散列

【答案】：A

【解析】：这是哈希表防死循环的一个定理：当使用二次探测法且表长 M 是一个形如 $4j+3$ 的素数时，只要哈希表的装填因子 $\alpha \leq 0.5$ (即表中至少一半是空的)，就能保证新元素必定能够找到空位置，不会出现探测无限循环无法插入的情况

19. 【考点：最小生成树】已知一个带权无向连通图，所有边的权值均不相同。关于该图的最小生成树 (MST)，下列断言错误的是 ()

A. 该图的最小生成树必定是唯一的
 B. 图中权值最小的边必定在最小生成树中
 C. 若向图的最小生成树中随意添加一条图中的边，必定会构成一个且仅一个环
 D. 图中权值最大的边必定不在最小生成树中

【答案】：D

【解析】：A 正确：边权各不相同，MST 唯一。B 正确：权最小边必然被选中（Kruskal 算法的第一步）。C 正确：树加一条边必成环。D 错误：如果这条最大权的边是连接图的两个连通分量的唯一桥梁（割边），那么无论它的权值多大，都必须把它加入 MST 以保证图的连通性

20. 【考点：图的遍历与生成树属性】给定一个无向连通图 $G = (V, E)$ ，从某顶点 v 出发对其分别进行深度优先搜索（DFS）和广度优先搜索（BFS），生成相应的深度优先生成树 T_{dfs} 和广度优先生成树 T_{bfs} 。已知图 G 中存在某条边 $e = (x, y)$ ，且它不是这两种生成树中的树边（即它是一条非树边）。下列关于非树边 e 在树中相对位置属性的论述中，必定错误的是（ ）

- A. 在 T_{dfs} 中， x 和 y 必定处于同一分支上，且其中一个是另一个的祖先
- B. 在 T_{bfs} 中， x 和 y 在树中的层数之差的绝对值必定不超过 1
- C. 在 T_{dfs} 中，有可能 x 和 y 位于树的两个完全不同且无祖先-后代关系的分支上（即 e 是一条跨越分支的“交叉边”）
- D. 若图 G 为完全图，则无论以哪个顶点为起点， T_{dfs} 必定为一条单链，而 T_{bfs} 的高度（规定根结点所在为第 1 层）必定为 2

【答案】：C

【解析】：

A 选项（正确）和 C 选项（错误）分析：

在无向图的深度优先搜索（DFS）中，非树边只可能是回边（Back-edge），即连接祖先与后代的边，绝对不可能存在交叉边（Cross-edge）

逻辑反证法：假设 T_{dfs} 中存在一条交叉边 (x, y) ，说明 x 和 y 属于两个不同分支，无祖先-后代关系。假设在 DFS 过程中先访问到 x （此时 y 尚未被访问），因为图是无向的，DFS 在退出 x 结点之前，必定会扫描到所有与 x 相连的边。当扫描到边 (x, y) 时，因为 y 尚未被访问，DFS 会立刻沿着这条边去访问 y ，从而使得 y 成为 x 的子孙结点。这与“无祖先-后代关系”产生矛盾。因此，C 选项描述的情况在无向图中绝不可能发生，C 必定错误；A 必然正确。（注：有向图的 DFS 树中是可以存在交叉边的）

B 选项分析（正确）：

在广度优先搜索（BFS）中，图的非树边只能连接同一层的结点，或者连接相邻两层的结点

数学证明：BFS 的本质是求无权图的最短路径。设根结点到 x 的最短距离为 $d(x)$ ，到 y 的最短距离为 $d(y)$ 。既然图中存在边 (x, y) ，根据最短路径的三角不等式，必定有 $d(y) \leq d(x) + 1$ 且 $d(x) \leq d(y) + 1$ 。由此推导出 $|d(x) - d(y)| \leq 1$ 。因此层数之差不可能超过 1

D 选项分析（正确）：

如果是完全图（任意两个顶点间都有直接相连的边）：

DFS：每访问一个顶点，总是能找到一个未访问过的邻居顶点一直深搜下去，直到所有顶点被访问完，因此 T_{dfs} 会形成一条包含所有顶点的单链（哈密顿路径）

BFS：从起点开始，第一步就会发现所有其他的 $n-1$ 个顶点，这 $n-1$ 个顶点会全部作为起点的直接孩子。因此 T_{bfs} 只有根（第 1 层）和它的所有孩子（第 2 层），高度恒为 2

21. 1. 算法设计思想：

判断完全二叉树的核心在于“结点必须连续，中间不能有空洞”。可以借助于层序遍历（使用队列）来实现

将根结点入队

在循环中依次将队头结点出队。无论其左右孩子是否为空，都将其左右孩子入队（若没有孩子，则把 NULL 指针入队）

当第一次出队遇到 NULL 指针时，停止遍历子结点

接下来检查队列中剩余的元素。如果剩余元素全为 NULL，则该树是完全二叉树；如果队列中还有非空的结点，说明在空结点之后还出现了有效结点，这违背了完全二叉树的定义，返回 false

2. C/C++ 代码实现：


```

typedef struct TreeNode {
    int data;
    struct TreeNode *left;
    struct TreeNode *right;
} TreeNode;

#define MAX_SIZE 1000 // 假设树结点数不超过 1000

bool IsCompleteBinaryTree(TreeNode* root) {
    if (root == NULL) return true; // 空树也是完全二叉树

    TreeNode* queue[MAX_SIZE];
    int front = 0, rear = 0;

    queue[rear++] = root; // 根节点入队

    while (front < rear) {
        TreeNode* current = queue[front++];

        if (current != NULL) {
            // 无论孩子是否为空，统统入队
            queue[rear++] = current->left;
            queue[rear++] = current->right;
        } else {
            // 遇到第一个 NULL 节点，结束扩展阶段
            break;
        }
    }

    // 检查队列剩余元素是否全是 NULL
    while (front < rear) {
        if (queue[front++] != NULL) {
            return false; // NULL 之后又出现了非空节点，非完全二叉树
        }
    }

    return true;
}

```

3. 复杂度分析：

时间复杂度：每个结点仅入队、出队一次，总遍历次数为 $O(N)$ (N 为树中结点总数)

空间复杂度：使用了辅助队列，最坏情况下队列需要容纳二叉树最底层的全部结点（即 $O(N)$ 级别），因此空间复杂度为 $O(N)$

22. 1. 算法设计思想：

本题的核心考点是树的图论等价定义：一个无向图 G 是一棵树的充要条件是—— G 必须是连通图，且包含 n 个顶点和 $n-1$ 条边。基于此，我们可以利用深度优先搜索（DFS）或广度优先搜索（BFS）来设计算法，具体步骤如下：

设定两个计数器 `v_count` 和 `e_count`，分别用于记录遍历过程中访问到的顶点数和边数（在邻接表中，无向图的每条边会被扫描两次）

从图的第 0 号顶点开始进行一次 DFS 遍历。在遍历过程中，每访问一个新顶点，`v_count` 加 1；每扫描到一条依附于该顶点的边（无论对端的邻接点是否被访问过），`e_count` 加 1

一次 DFS 结束后，如果该图是连通的，必然能访问到所有的顶点；如果它是一棵树，其边数必然为 $n-1$ 。由于无向图邻接表中的一条边对应两个弧结点，因此 `e_count` 最终的值应当

为 $2(n-1)$

最后, 判断若 `v_count == G.vexnum` 且 `e_count == 2 * (G.vexnum - 1)`, 则该图是一棵树, 返回 `true`; 否则返回 `false`

2. 图的邻接表存储结构定义:

```
#define MAX_VERTEX_NUM 100 // 图中顶点数目的最大值

// 边（弧）表结点
typedef struct ArcNode {
    int adjvex; // 该弧所指向的顶点的位置
    struct ArcNode *nextarc; // 指向下一条弧的指针
} ArcNode;

// 顶点表结点
typedef struct VNode {
    int data; // 顶点信息
    ArcNode *firstarc; // 指向第一条依附该顶点的弧的指针
} VNode, AdjList[MAX_VERTEX_NUM];

// 图的定义
typedef struct {
    AdjList vertices; // 邻接表
    int vexnum, arcnum; // 图的当前顶点数和弧数
} ALGraph;
```

3. C/C++ 代码实现:

```
// 全局变量或外部变量, 用于记录状态和计数
bool visited[MAX_VERTEX_NUM];
int v_count = 0; // 记录遍历的顶点数
int e_count = 0; // 记录遍历的边数（以弧的数量计）

// 基于 DFS 遍历图并进行统计
void DFS_Count(ALGraph G, int v) {
    visited[v] = true; // 标记当前顶点已访问
    v_count++; // 访问的顶点数加 1

    // 遍历顶点 v 的所有邻接点
    ArcNode *p = G.vertices[v].firstarc;
    while (p != NULL) {
        e_count++; // 每扫描到一条边, 边数加 1（无向图一条边会在此处被统计两次）
        int w = p->adjvex;

        // 若邻接点未被访问, 则递归深搜
        if (!visited[w]) {
            DFS_Count(G, w);
        }
        p = p->nextarc; // 继续检查下一个邻接点
    }
}

// 判断无向图 G 是否为一棵树
bool IsTree(ALGraph G) {
    // 鲁棒性检查: 空图算作树（依具体约定而定, 通常 408 中顶点数 > 0）
    if (G.vexnum == 0) return false;
```



```

// 1. 初始化访问标记数组和计数器
for (int i = 0; i < G.vexnum; i++) {
    visited[i] = false;
}
v_count = 0;
e_count = 0;

// 2. 从 0 号顶点启动一次 DFS 遍历
DFS_Count(G, 0);

// 3. 校验树的充要条件
// 条件1: v_count == G.vexnum 说明图是连通的 (一次 DFS 就能访问完所有顶点)
// 条件2: e_count == 2 * (G.vexnum - 1) 说明图正好有 n-1 条边
if (v_count == G.vexnum && e_count == 2 * (G.vexnum - 1)) {
    return true;
} else {
    return false;
}
}

```

4. 复杂度分析:

时间复杂度: 算法本质上是对图进行了一次标准的深度优先搜索。在 DFS 过程中, 访问了该连通分量内的所有顶点和边。因此, 最坏情况下的时间复杂度为 $O(|V| + |E|)$, 其中 $|V|$ 为图的顶点数, $|E|$ 为图的边数

空间复杂度: 算法使用了一个大小为 $|V|$ 的辅助数组 `visited`; 此外, DFS 递归调用会消耗系统栈空间, 最坏情况下 (如图本身退化成一条单链), 递归深度为 $|V|$ 。因此, 总的空间复杂度为 $O(|V|)$

23. 1. 构造哈希表填表过程:

12: $H(12) = 1$, 地址 1 空, 存入 (探测 1 次)
 23: $H(23) = 1$, 冲突。 $H_1 = (1+1) \% 11 = 2$, 空, 存入 (探测 2 次)
 45: $H(45) = 1$, 冲突。探 2 冲突, 探 3 空, 存入 (探测 3 次)
 57: $H(57) = 2$, 冲突。探 3 冲突, 探 4 空, 存入 (探测 3 次)
 20: $H(20) = 9$, 地址 9 空, 存入 (探测 1 次)
 03: $H(03) = 3$, 冲突。探 4 冲突, 探 5 空, 存入 (探测 3 次)
 78: $H(78) = 1$, 冲突。依次探 1,2,3,4,5 均冲突, 探 6 空, 存入 (探测 6 次)
 31: $H(31) = 9$, 冲突。探 10 空, 存入 (探测 2 次)
 15: $H(15) = 4$, 冲突。依次探 4,5,6 冲突, 探 7 空, 存入 (探测 4 次)
 36: $H(36) = 3$, 冲突。依次探 3,4,5,6,7 冲突, 探 8 空, 存入。 (探测 6 次)

最终哈希表状态表:

地址	0	1	2	3	4	5	6	7	8	9	10
关键字		12	23	45	57	3	78	15	36	20	31
探测次数		1	2	3	3	3	6	4	6	1	2

2. 查找成功的 ASL 计算:

$$\begin{aligned}
 ASL &= \frac{\text{各元素探测次数之和}}{\text{元素总个数}} \\
 &= \frac{1 + 2 + 3 + 3 + 1 + 3 + 6 + 2 + 4 + 6}{10} = \frac{31}{10} = 3.1
 \end{aligned}$$

3. 查找失败的 ASL 计算:

查找失败意味着按 $H(\text{key}) = 0 \dots 10$ 计算出初始地址, 然后顺推直到遇到空槽 (表中 0 地址为空)

初始映射在 0: 直接为空, 1 次

初始映射在 1: 历经 1,2,3,4,5,6,7,8,9,10,0 才找到空 (需绕回 0), 共 11 次

初始映射在 2: 历经 2,3,4,5,6,7,8,9,10,0, 共 10 次

初始映射在 3: 共 9 次

初始映射在 4: 共 8 次

初始映射在 5: 共 7 次

初始映射在 6: 共 6 次

初始映射在 7: 共 5 次

初始映射在 8: 共 4 次

初始映射在 9: 历经 9,10,0, 共 3 次

初始映射在 10: 历经 10,0, 共 2 次

$$\text{ASL} = \frac{1 + 11 + 10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2}{11} = \frac{66}{11} = 6$$

24. 初始化: S 集合只包含起点 {0}

初始 `dist` 数组为第 0 行的值: `[0, 10, ∞, 30, 100]`

初始 `path` 数组: `[-1, 0, -1, 0, 0]`

第 1 步:

在 $V-S$ 集合中找 `dist` 最小的值。当前最小为 `dist[1] = 10`

将顶点 1 并入集合 S 。当前 $S = \{0, 1\}$

以 1 为中间点更新 `dist`:

1 的出边有 1→2 (50)

`dist[0] + matrix[1][2] = 10 + 50 = 60 < ∞`, 更新 `dist[2] = 60`, `path[2] = 1`

本次迭代后状态:

`dist: [0, 10, 60, 30, 100]`

`path: [-1, 0, 1, 0, 0]`

第 2 步:

在 $V-S = \{2, 3, 4\}$ 中找 `dist` 最小值。当前最小为 `dist[3] = 30`

将顶点 3 并入集合 S 。当前 $S = \{0, 1, 3\}$

以 3 为中间点更新 `dist`:

3 的出边有 3→2 (20), 3→4 (60)

对 2: `dist[3] + matrix[3][2] = 30 + 20 = 50 < 60`, 更新 `dist[2] = 50`, `path[2] = 3`

对 4: `dist[3] + matrix[3][4] = 30 + 60 = 90 < 100`, 更新 `dist[4] = 90`, `path[4] = 3`

本次迭代后状态:

`dist: [0, 10, 50, 30, 90]`

`path: [-1, 0, 3, 0, 3]`

第 3 步:

在 $V-S = \{2, 4\}$ 中找 dist 最小值。当前最小为 $\text{dist}[2] = 50$

将顶点 2 并入集合 S 。当前 $S = \{0, 1, 3, 2\}$

以 2 为中间点更新 dist :

2 的出边有 2→4 (10)

$\text{dist}[2] + \text{matrix}[2][4] = 50 + 10 = 60 < 90$, 更新 $\text{dist}[4] = 60$, $\text{path}[4] = 2$

本次迭代后状态:

$\text{dist}: [0, 10, 50, 30, 60]$

$\text{path}: [-1, 0, 3, 0, 2]$

第 4 步:

只剩下顶点 4, 将其并入 S 。 $S = \{0, 1, 3, 2, 4\}$ 。算法结束

最终结果汇总:

从顶点 0 到各顶点的最短路径及长度:

到 1: 距离 10, 路径 0 → 1

到 2: 距离 50, 路径 0 → 3 → 2 (由 $\text{path}[2]=3$, $\text{path}[3]=0$ 倒推)

到 3: 距离 30, 路径 0 → 3

到 4: 距离 60, 路径 0 → 3 → 2 → 4 (由 $\text{path}[4]=2$, $\text{path}[2]=3$, $\text{path}[3]=0$ 倒推)